Virtuellt minne

Jimmy Åhlander

2025-11-27

Mittuniversitetet
MID SWEDEN UNIVERSITY

Agenda

- Virtuellt minne:
  - Fortsättning på paging.
- Demand paging:
  - Att hämta pages vid behov.
  - Pager.
  - Sidfel (*page faults*).
- Sidersättning (*page replacement*):
  - Vad händer när det fysiska minnet är fullt?
  - Sidersättningsalgoritmer (*page replacement algorithms*).

Föreläsningen i ett nötskal

- Den här föreläsningen utökar det fysiska minnet genom att introducera virtuellt minne.

- Föreläsningen motsvarar kapitel 10 *Virtual Memory* i Operating System Concepts.

Fortsättning på paging …

Obesvarade frågor sedan vi introducerade paging:

- Vad händer om vi vill köra ett program som är större än det fysiska minnet?
- Behöver vi alltid ladda in hela programkoden?
  - Vissa delar av ett program används mycket sällan, exempelvis felhantering.

- Processens minne är nu uppdelat i sidor. En idé är då att ladda in endast de sidor som processen faktiskt behöver.

- Detta är idén bakom **virtuellt minne** och **demand paging**.

Virtuellt minne

- Den fysiska och de logiska adressrymderna har separerats med paging.
- Nu utökar vi konceptet genom att introducera virtuellt minne.
- Virtuellt minne innebär att vi låtsas som om vi har ett i det närmaste oändligt stort minne – det virtuella minnet.

Paging med backing store

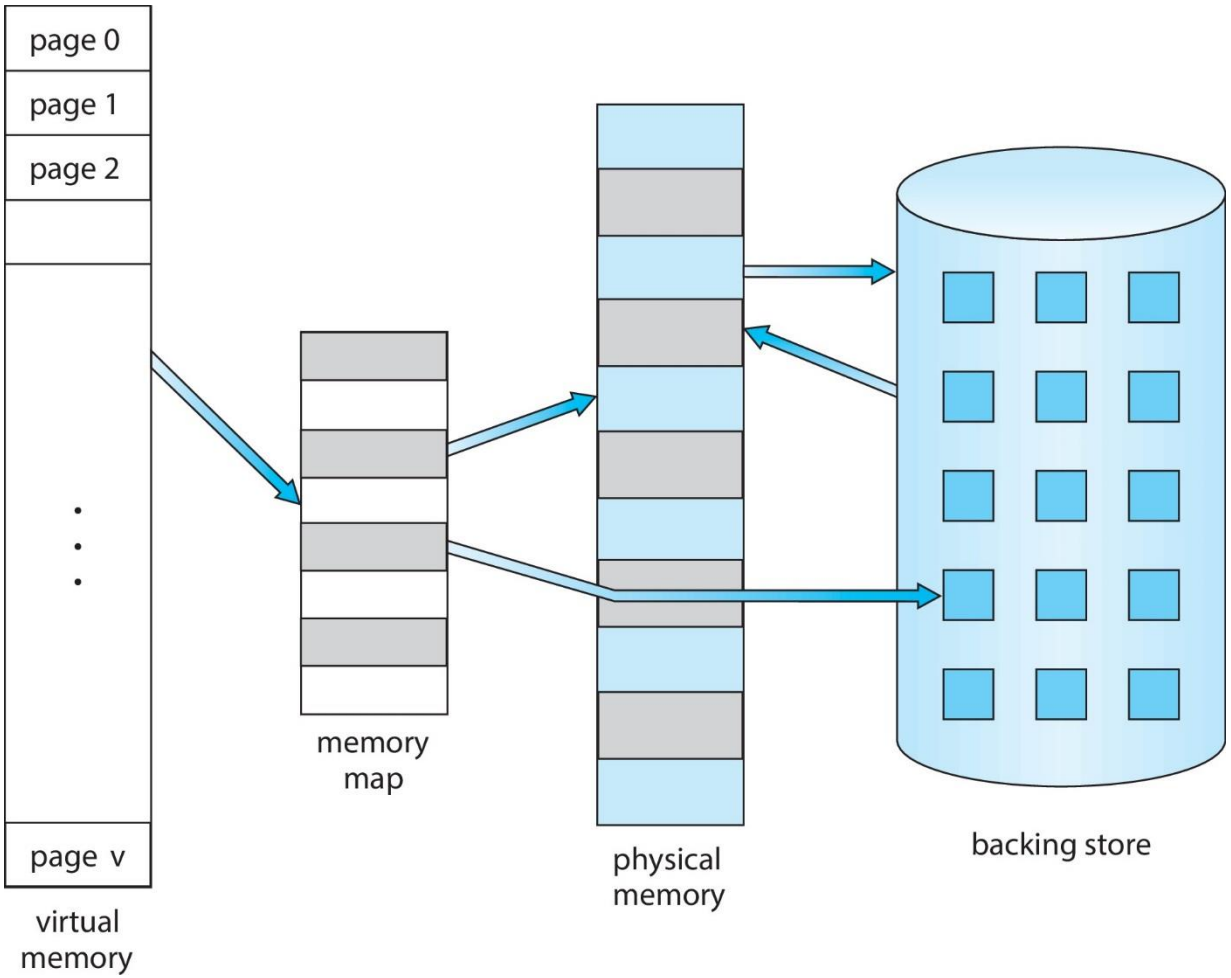

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.01

Virtuellt minne

- Med den här approachen har varje process tillgång till hela den virtuella minnesrymden. Så vitt processen vet har den tillgång till precis varje logisk adress.

Virtuellt minne

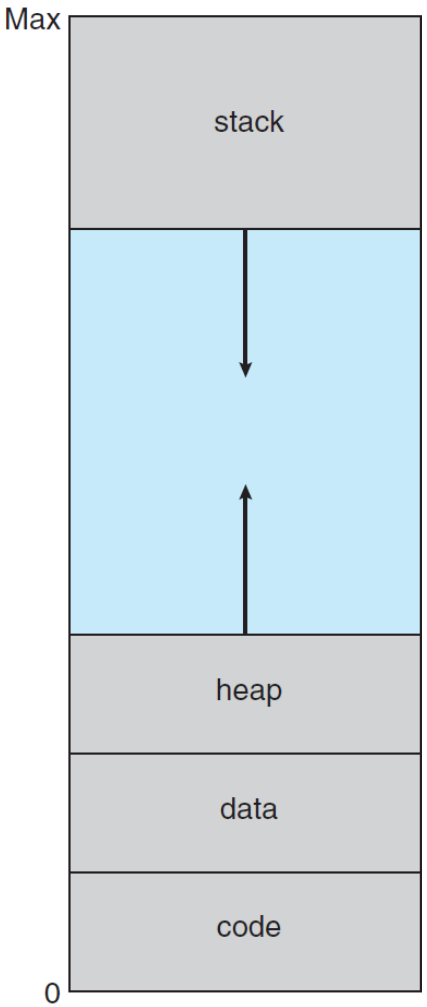

Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 9.02

Virtuellt minne

- Med ett enormt tomrum kan vi nu inkludera en enorm stack och en enorm heap.
- Vi kan också använda tomrummet för andra ändamål, exempelvis delade sidor.

Virtuellt minne

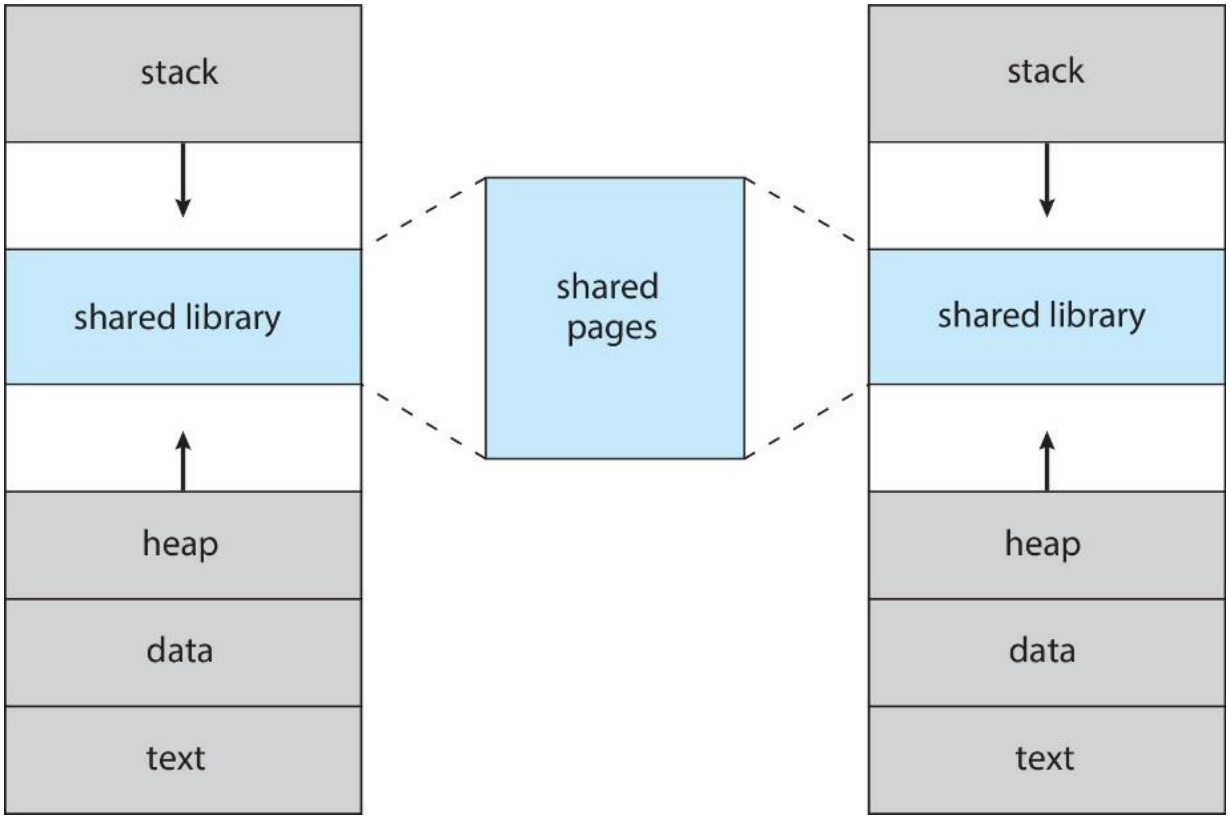

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.03

Swapping vs. demand paging

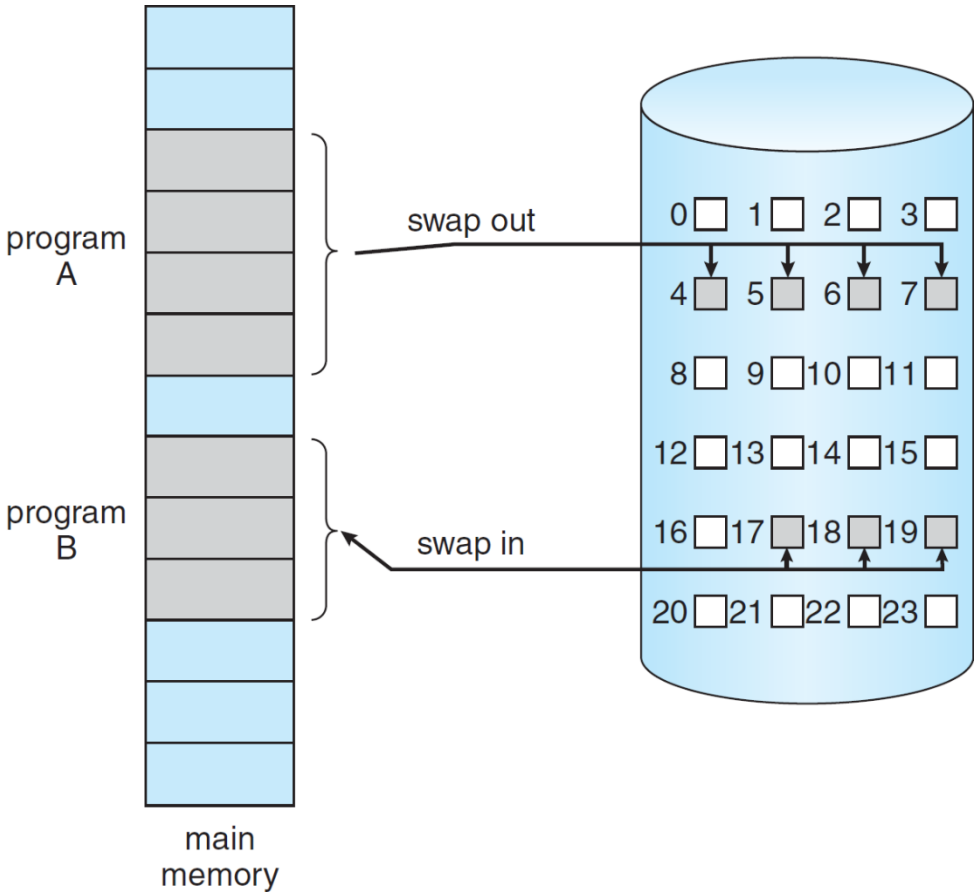

Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 9.04

Demand paging
Swapper vs. Pager

- En swapper arbetar med hela processer.
- Vad får vi när vi ”swappar” in och ut endast delar av en process?
  - En *lazy swapper*.

- Med paging används en **pager** för att ladda in och ut sidor från en process.

- En pager kräver hårdvarustöd för att kunna veta vilka sidor som behövs men som inte laddats in till minnet. Vi använder valid-invalid bitarna som introducerades tidigare.

Demand paging

Några sidor i primärminnet och några på disk

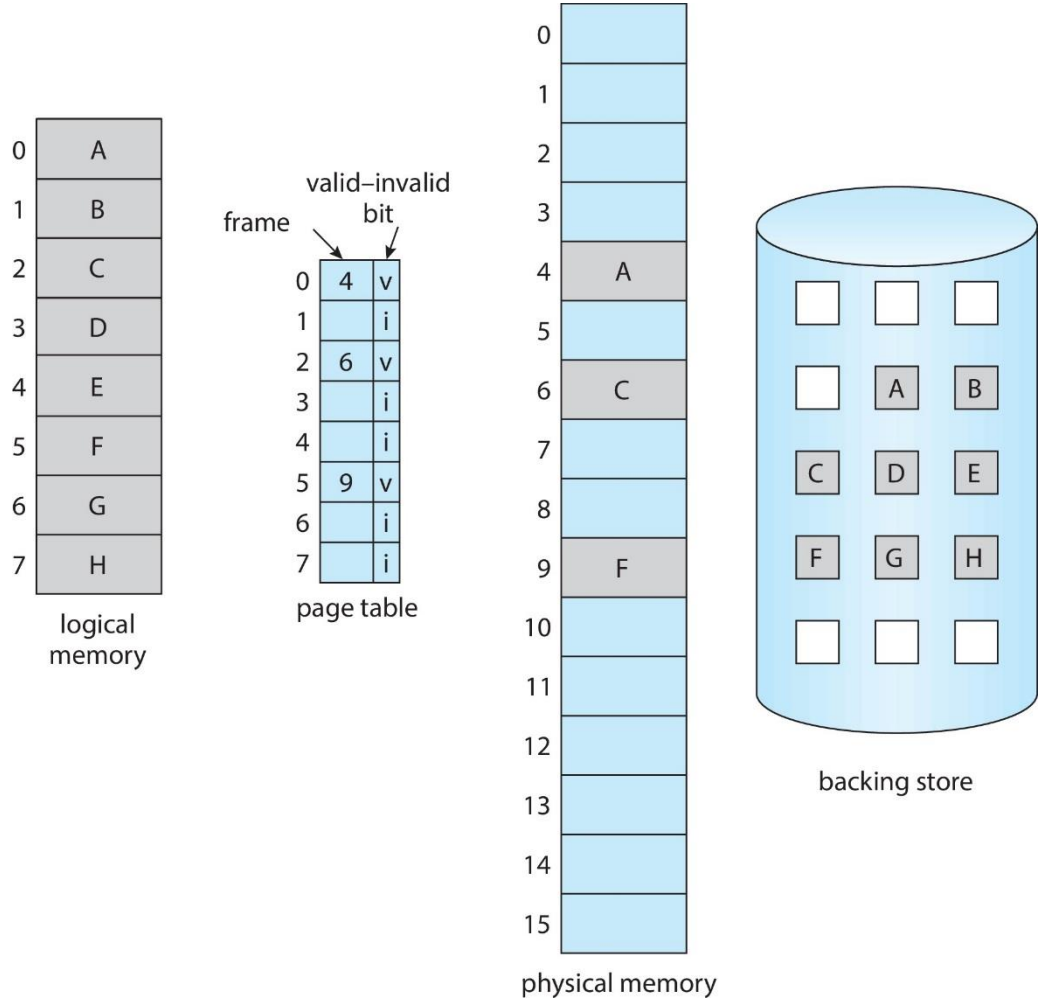

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.04

Demand paging

Sidfel (*page fault*)

- Så länge alla sidor en process behöver är allokerade till ramar i minnet fungerar allt precis som tidigare, men …
- Vad händer när en process refererar till en sida som inte är inladdad i minnet?
- MMU:n genererar en trap till operativsystemet.
  Specifikt: Ett **sidfel**.

Demand paging

Sidfel (*page fault*)

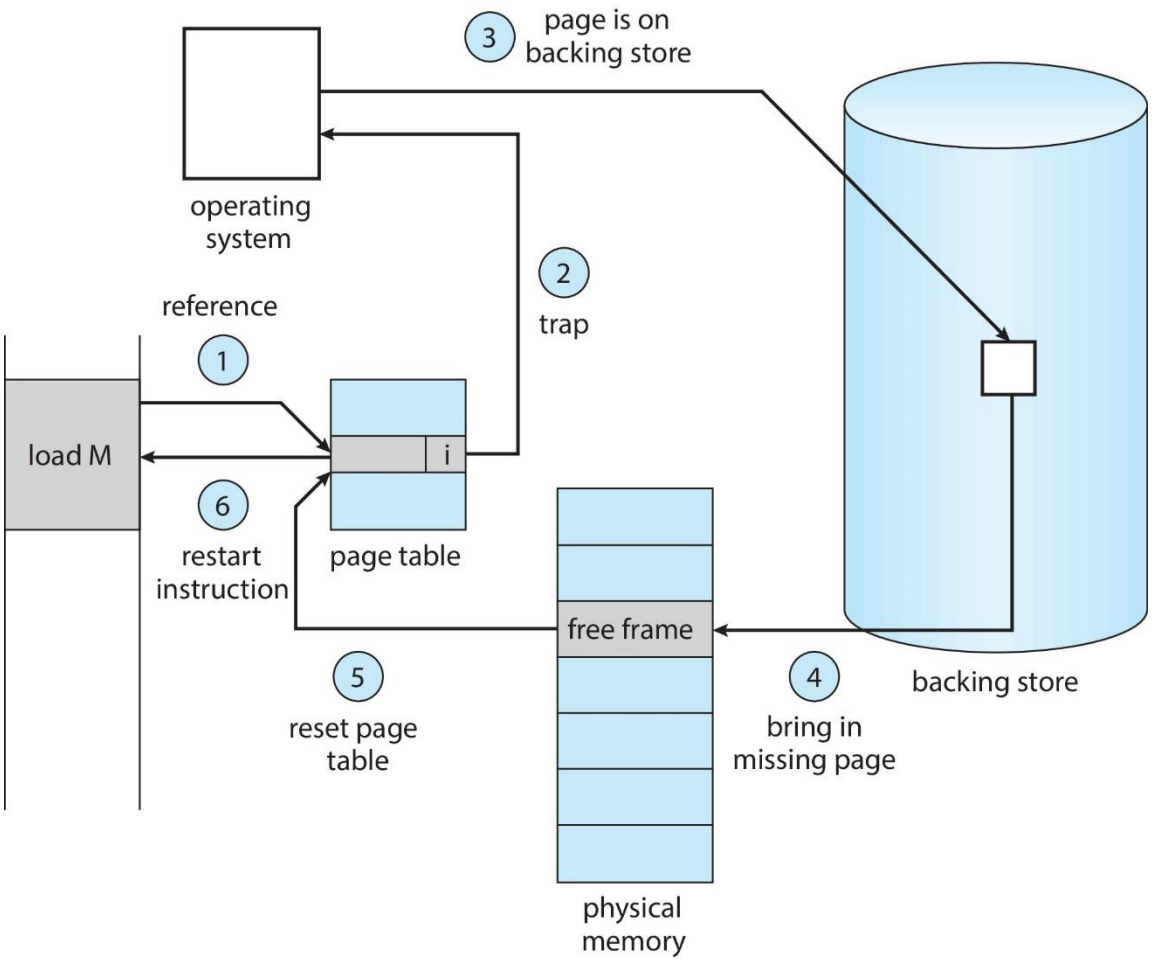

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.05

Äkta demand paging

- Äkta (*pure*) demand paging innebär att vi aldrig laddar in en sida till primärminnet till dess att den behövs (genom att generera ett sidfel).
- Hårdvarukraven för demand paging är mestadels samma som för paging med valid-invalid bitar.
- Demand paging kräver emellertid också möjligheten att starta om instruktioner i CPU:n efter ett sidfel.

Demand paging
Prestanda

Vilken prestanda har demand paging?

- En typisk minnesåtkomsttid är mellan 10 och 200 nanosekunder.

- Om inga sidfel uppstår så förblir prestandan densamma som tidigare.

- Det går att beräkna den effektiva åtkomsttiden på samma sätt som tidigare med paging och en TLB.

Demand paging

Effektiv åtkomsttid

Effective access time

The effective access time t is

$$t = (1 - p) \times t_m + p \times t_p,$$

where p is the probability of a page fault, t_m is the memory access time and $t_p > t_m$ is the page-fault time.

Demand paging

Effektiv åtkomsttid - exempel

Example

We have $t_m = 200\ \mathrm{ns}$ and $t_p = 8\ \mathrm{ms} = 8000000\ \mathrm{ns}$. We want $t < 220\ \mathrm{ns}$, i.e. less than 10 percent loss.

$$
\begin{aligned}
t &> (1-p) \times t_m + p \times t_p && (1)\\
220 &> (1-p) \times 200 + p \times 8000000 && (2)\\
220 &> 200 - p \times 200 + p \times 8000000 && (3)\\
220 &> 200 + p \times (8000000 - 200) && (4)\\
220 &> 200 + 7999800 \times p && (5)\\
20 &> 7999800 \times p && (6)\\
\frac{20}{7999800} \approx 0.0000025 &> p && (7)
\end{aligned}
$$

Sidersättning

Vad händer när det fysiska minnet är fullt och ett sidfel genererats?

- Vi måste välja en ram vars sida offras.
- Det finns olika sidersättningsalgoritmer för detta.

Vi måste även se upp för *thrashing*.

– Konstant paging.

Thrashing

När vi spenderar mer tid på paging än att exekvera instruktioner.

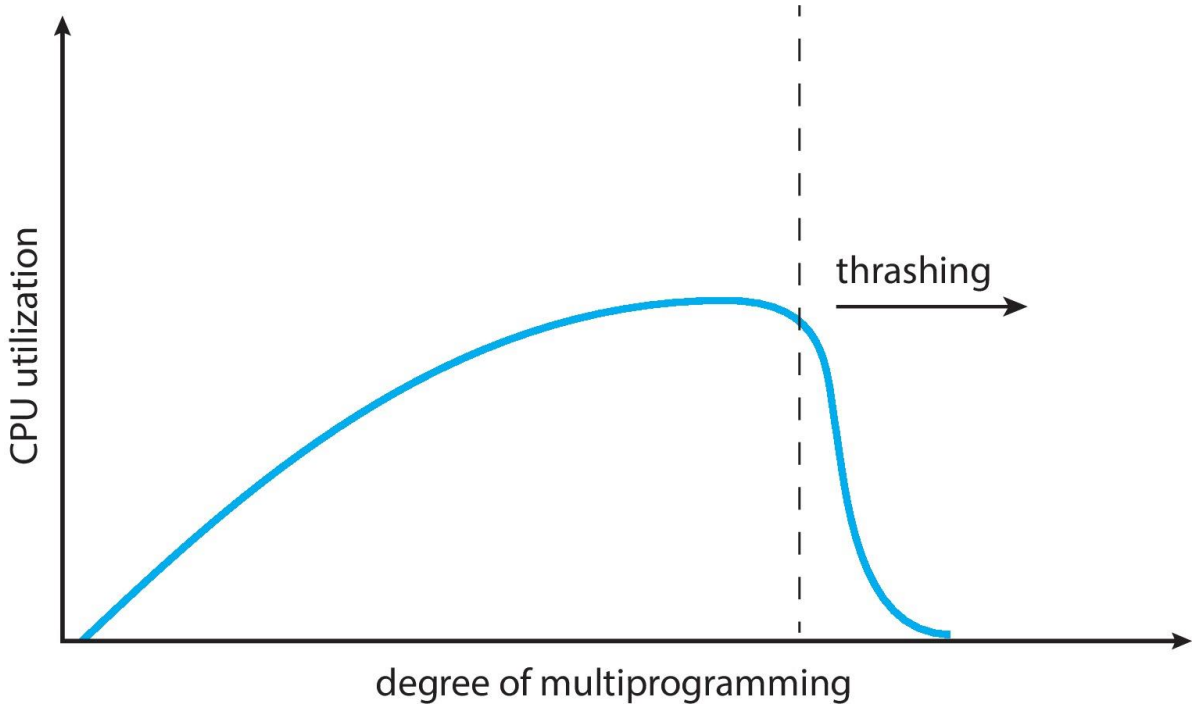

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.20

Sidersättning

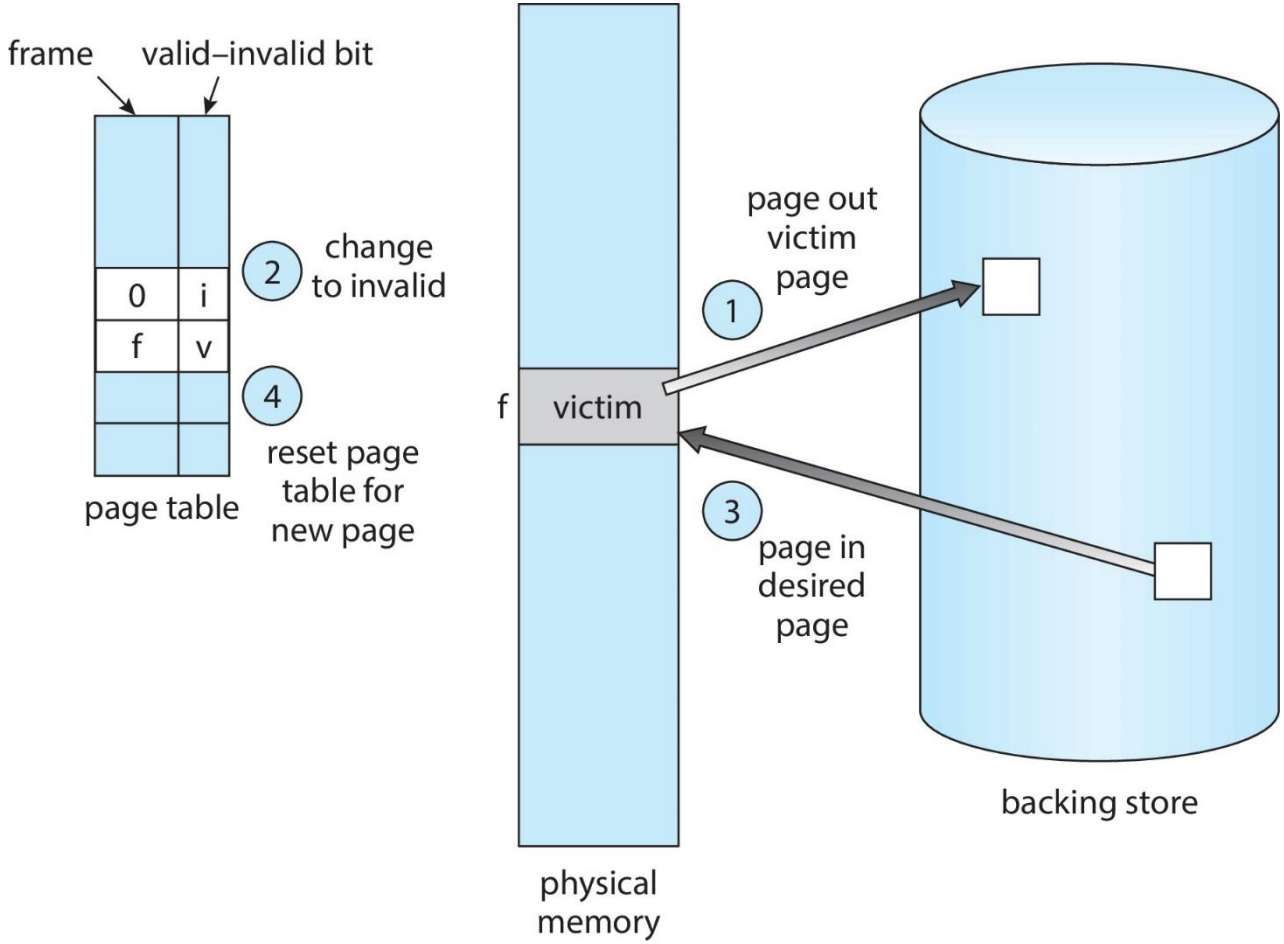

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 10.10

Sidersättningsalgoritmer

Kriterier:

- Vi vill minimera antalet sidfel.
  - Genom att offra sidor vi förhoppningsvis inte kommer behöva snart igen minimeras antalet sidfel.
- Vissa sidor är dyrare att byta ut än andra.
  - Om sidans innehåll modifierats måste sidan skrivas tillbaks till disk innan den kan offras.
  - Om sidans innehåll är omodifierat kan den istället skrivas över direkt.

Sidersättningsalgoritmer

Algoritmer:

- First in, first out (FIFO) page replacement.
- Optimal page replacement.
- Least-recently used (LRU) page replacement.
- LRU-approximationer:
  - Second chance.
  - Enhanced second chance.

Det finns fler algoritmer …

Sidersättningsalgoritmer
First in, first out page replacement

- Ersätter sidor i FIFO-ordning.
- Ramen som innehåller den äldsta sidan kastas.
  - Den kom in först och kastas därför först.

+ Kräver ingen specifik hårdvara.
- Tar inte hänsyn till att sidor snart kan komma att användas igen.
- Särskiljer inte på modifierade och icke-modifierade sidor.

Sidersättningsalgoritmer
Optimal page replacement

- Ersätter sidor för att på ett optimalt sätt minimera antalet sidfel.
- Ramen som innehåller den sida som inte kommer att behövas i framtiden kastas.
  - Om samtliga sidor i ramarna kommer att behövas någon gång igen kastas den som behövs sist i sekvensen.

+ Den är optimal teoretiskt och genererar det minimala antalet sidfel.
- Kräver insyn i framtiden. I simulationer är referenssträngen känd i förväg, vilket inte speglar verkliga scenarion.
- Särskiljer inte på modifierade och icke-modifierade sidor. Antalet sidfel är inte det enda som påverkar tidskostnaden.

Sidersättningsalgoritmer
Least-recently used page replacement

- Använder vetskap om hur ofta sidorna refererats hittills för att offra den sida som mest troligen inte kommer behövas i framtiden.
- Varje sida har en anknuten räknare. Varje gång sidan blir refererad till ökar räknaren. Samtliga ramar är placerade i en cirkulär kö. Varje gång en sida ska offras går algoritmen genom kön. Varje sida som passeras får sin räknare sänkt. Stöter algoritmen på en sida där räknaren är 0 offras sidan.

+ Sänker antalet sidfel jämfört med FIFO genom att behålla sidor som använts ofta. Detta då de troligen kommer behövas snart igen.
- Dyrt att spåra exakt hur många gånger varje sida blivit refererad till. Hur många extra bitar per ram i sidtabellen är rimligt?
- Särskiljer inte alltid på modifierade och icke-modifierade sidor. Olika varianter finns.

Sidersättningsalgoritmer
Second chance

- En billigare LRU-approximation.
- Varje sida har en anknuten bit: Andra chansen. Varje gång sidan blir refererad till sätts andra chansen. Samtliga ramar är placerade i en cirkulär kö. Varje gång en sida ska offras går algoritmen genom kön. Varje sida som passeras får sin andra chans nollställd. Stöter algoritmen på en sida som saknar andra chansen offras sidan.

+ Sänker antalet sidfel jämfört med FIFO genom att behålla sidor som använts ofta. Detta då de troligen kommer behövas snart igen.
- Särskiljer inte alltid på modifierade och icke-modifierade sidor.

Sidersättningsalgoritmer
Enhanced second chance

- En LRU-approximation som visar hänsyn till modifierade sidor.
- Varje sida har två anknutna bitar: Andra chansen och en mod bit. Varje gång sidan blir refererad till sätts andra chansen. Om sidan skrivs till sätts mod biten. Samtliga ramar är placerade i en cirkulär kö. Varje gång en sida ska offras går algoritmen genom kön i fler varv.

Varv:

1. Leta efter sidor som saknar båda bitar. Hittas en sida offras den.
2. Leta efter sidor som saknar andra chansen men har mod bit. Stryk andra chanser som bemöts. Hittas en matchande sida offras den.
3. Upprepa steg 1.
4. Offra sidan vid huvudpekaren till kön.

\+ Sänker antalet sidfel jämfört med FIFO genom att behålla sidor som använts ofta. Detta då de troligen kommer behövas snart igen.

\+ Sänker tiden att hantera sidfel då valda offer ofta kan skrivas över direkt.

\- Kan behöva iterera flera gånger över alla ramar för att hitta ett offer.

Virtuellt minne
Sammanfattning

- Med virtuellt minne är vi inte längre begränsade till det fysiska minnet.
- Med demand paging minskar vi belastningen på det fysiska minnet.
- Sidfel ökar minnesåtkomsttiden ytterligare.
- När det fysiska minnet blir fullt måste sidor offras. Offren måste välja omsorgsfullt för att minimera prestandakostnaden.